KANT AX/AI 엔지니어 트랙 · 보충 학습 자료

# 파이썬 기초 4
# 딕셔너리

번호 대신 키로
값을 찾고 바꾸기



# 오늘의 순서


1 **만들고 꺼내기** — 4–9쪽
번호 대신 키, 메뉴판 모양, { 키: 값 }, [키]와 KeyError, get

2 **바꾸고 지우기** — 10–13쪽
추가와 수정, 겹치지 않는 키, del과 pop, 키를 보는 in

3 **둘러보기** — 14–19쪽
keys, values, items, list(), for로 한 쌍씩, sum과 sorted, get으로 세기

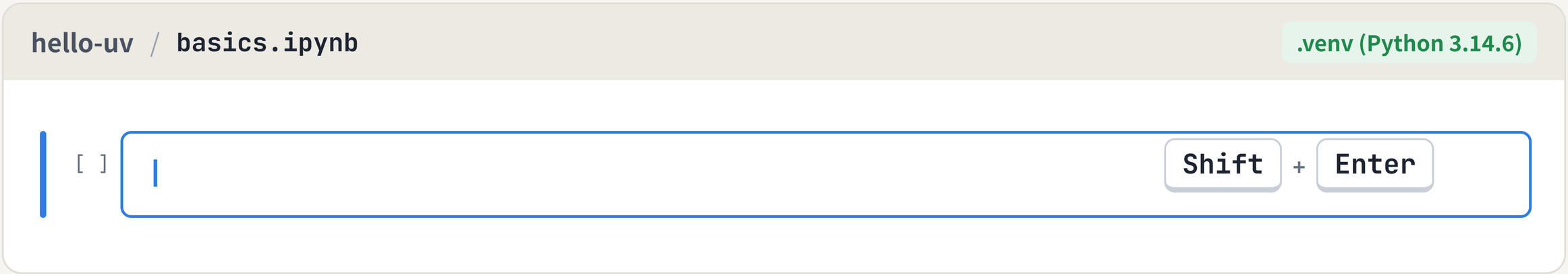
4 **리스트와 섞기** — 20–22쪽
값이 리스트인 딕셔너리, 딕셔너리 리스트, for로 주문 돌기

5 **실제로 써 보기** — 23–27쪽
리스트·튜플과 비교, 오류 세 가지, 영수증, 장바구니, 판매 개수


3쪽은 노트북 준비, 28–29쪽은 막힐 때와 정리예요.

들어가기

# 14에서 쓰던 노트북에 이어서 적어요




1. VS Code로 hello-uv 폴더를 열어요
2. 탐색기에서 basics.ipynb를 열어요
3. 오른쪽 위 커널이 .venv인지 봐요
4. 셀에 적고 Shift+Enter로 실행해요

---

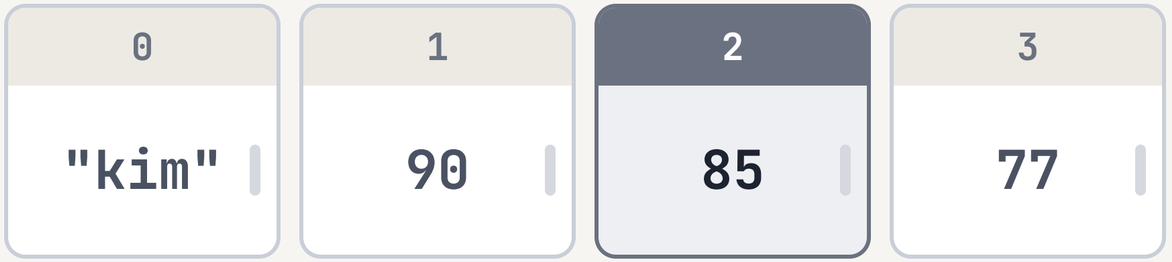
기억이 안 나면 먼저 다시 봐요. 노트북과 커널 **09 · 25–27쪽** · f-string, 리스트, for **13** · in, 2차원 리스트 **14**

1. 만들고 꺼내기

# 번호 대신 키로 찾아요

[1]
```python
student = ["kim", 90, 85, 77]
print(student[2])
```

```
85
```




2번이 영어라는 걸 외워야 해요

[2]
```python
student = {"name": "kim", "kor": 90,
           "eng": 85, "math": 77}
print(student["eng"])
```

```
85
```

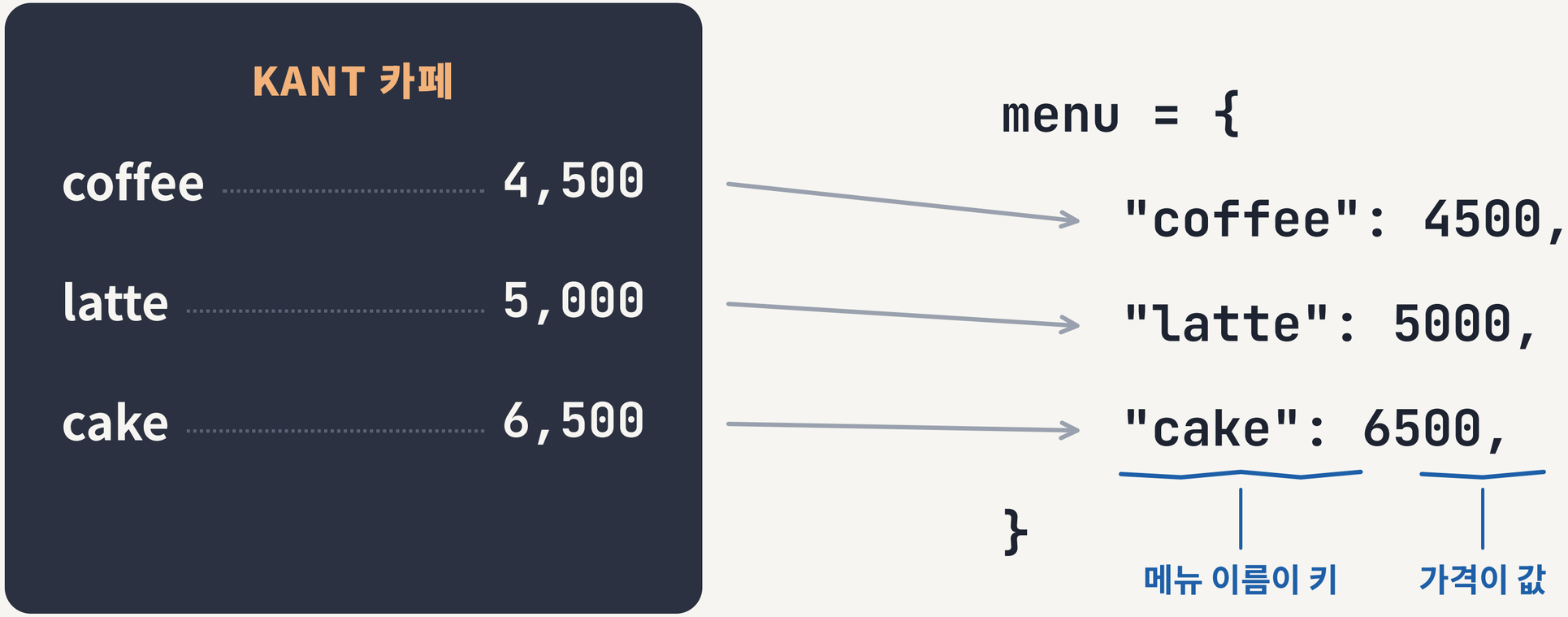



읽기만 해도 영어 점수예요

**문에 적힌 이름이 키(key), 안에 든 것이 값(value)이에요.**

1. 만들고 꺼내기

# 메뉴판이 바로 딕셔너리 모양이에요




이름으로 값을 찾는 표는 딕셔너리로 담을 수 있어요. 연락처는 **이름 → 전화번호**, 설정은 **항목 → 값**이에요.

# 중괄호 안에 키: 값을 쉼표로 이어요




[3]
```python
menu = {"coffee": 4500, "latte": 5000, "cake": 6500}
print(menu)
print(type(menu))
print(len(menu))
cart = {}
print(cart)
print(len(cart))
```

```
{'coffee': 4500, 'latte': 5000, 'cake': 6500}
<class 'dict'>   ← dict는 dictionary(사전)의 줄임말
3   ← 키: 값 쌍이 3개
{}   ← 빈 딕셔너리
0   ← 쌍이 0개
```

menu

| "coffee" |
| --- |
| 4500 |

| "latte" |
| --- |
| 5000 |

| "cake" |
| --- |
| 6500 |

1. 만들고 꺼내기

# 대괄호에 키를 넣어 꺼내요

[4]
```python
print(menu["latte"])
price = menu["cake"]
print(price * 2)
```

```
5000
13000
```

← 꺼낸 값으로 바로 계산해요

[5]
```python
menu["tea"]
```

```
KeyError: 'tea'
```

← 찾던 키가 없어요

menu

| "coffee" | "latte" | "cake" |
| --- | --- | --- |
| 4500 | 5000 | 6500 |

1. 만들고 꺼내기

# get은 없는 키에도 오류 없이 답해요

[6]
```python
print(menu.get("latte"))
print(menu.get("tea"))
print(menu.get("tea", 0))
```

```
5000    ← latte가 있으니 그 값
None    ← tea가 없으면 None, 값이 없다는 뜻이에요
0       ← 없을 때 줄 값을 정해 둘 수 있어요
```

---

없는 키를 물었을 때

| | | |
|---|---|---|
| `menu["tea"]` | `KeyError: 'tea'` | 셀이 거기서 멈춰요 |
| `menu.get("tea", 0)` | `0` | 0을 받고 다음 줄로 넘어가요 |

1. 만들고 꺼내기

# get은 긴 코드를 한 줄로 줄인 거예요

회색 줄은 참고, 읽기만 해요. if는 17에서 배워요.

[7]
```python
print(menu["cakee"] if "cakee" in menu else 5000)
print(menu.get("cakee", 5000))
print(menu["cake"] if "cake" in menu else 5000)
print(menu.get("cake", 5000))
```

cakee는 키가 없어서 둘 다 5000

cake는 키가 있어서 둘 다 6500

```
5000
5000
6500
6500
```

[8]
```python
sale = {"cake": None}
print(sale.get("cake", 5000))
print(sale.get("cakee", 5000))
```

```
None    ← cake 키는 있어요. 값 None을 그대로 줘요
5000    ← 기본값은 키가 없을 때만 써요
```

2. 바꾸고 지우기

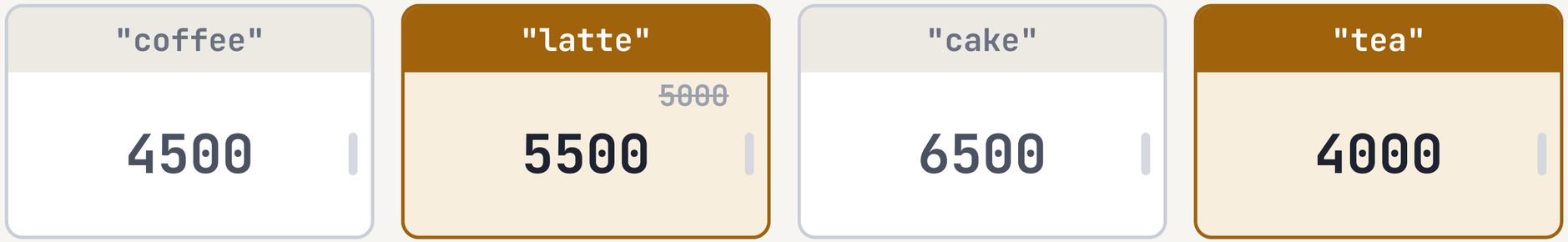
# 없는 키면 추가, 있는 키면 수정이에요

[1]
```python
menu = {"coffee": 4500, "latte": 5000, "cake": 6500}
menu["tea"] = 4000   ← tea 키가 없어서 새 칸이 생겨요
print(menu)
menu["latte"] = 5500   ← latte 키가 있어서 값만 바뀌어요
print(menu)
print(len(menu))
```

Python

```
{'coffee': 4500, 'latte': 5000, 'cake': 6500, 'tea': 4000}
{'coffee': 4500, 'latte': 5500, 'cake': 6500, 'tea': 4000}
4   ← 칸은 4개 그대로예요
```

menu



2. 바꾸고 지우기

# 키는 겹치지 않아요. 값은 겹쳐도 돼요

[2]
```python
dup = {"latte": 5000, "latte": 5500}
print(dup)
print(len(dup))
score = {"kim": 90, "lee": 90, "park": 90}
print(score)
```

```
{'latte': 5500}
1
{'kim': 90, 'lee': 90, 'park': 90}
```

dup




~~5000~~

먼저 넣은 값은 덮어써요

score




값은 같아도 키가 다르면 칸이 따로예요

2. 바꾸고 지우기

# del과 pop으로 지워요

[3]
```python
del menu["tea"]   ← 키와 값을 같이 지워요
print(menu)
price = menu.pop("cake")   ← 지우면서 값을 돌려줘요
print(price)
print(menu)
```

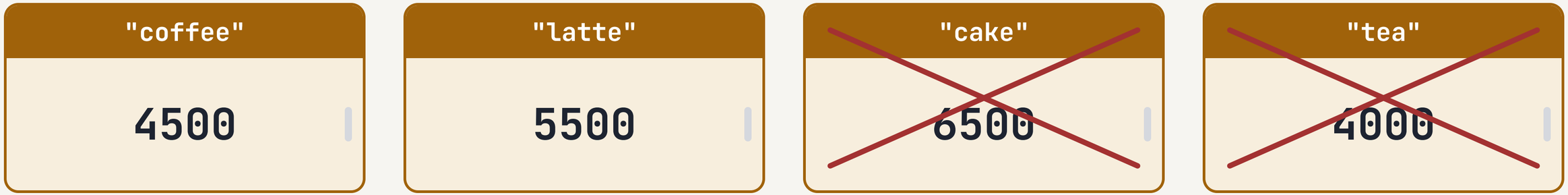
```
{'coffee': 4500, 'latte': 5500, 'cake': 6500}
6500
{'coffee': 4500, 'latte': 5500}
```

[4]
```python
del menu["tea"]
```

```
KeyError: 'tea'
```

없는 키를 지우면 꺼낼 때와 같은 오류예요

menu




**13의 리스트 pop은 번호를, 딕셔너리 pop은 키를 넣어요.**

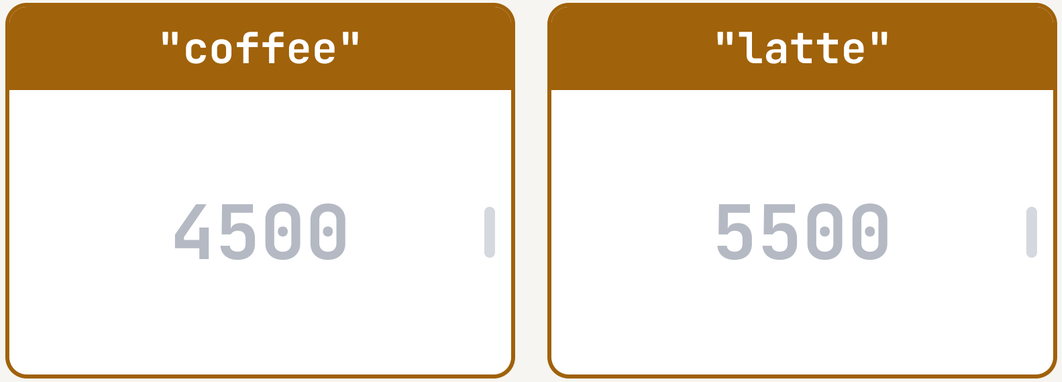
2. 바꾸고 지우기

# in은 키 쪽만 봐요

[5]
```python
print("latte" in menu)
print("tea" in menu)
print("tea" not in menu)
print(5500 in menu)
print(len(menu))
```

```
True    ← latte는 키예요
False   ← tea는 앞 장에서 지웠어요
True    ← not in은 반대로 답해요
False   ← 5500은 값이라 안 봐요
2       ← 남은 쌍은 2개
```

menu




값에서 찾으려면

```
5500 in menu.values()
```

values()는 3파트에서 봐요

3. 둘러보기

# keys, values, items로 모아 봐요

[1]
```python
menu = {"coffee": 4500, "latte": 5000, "cake": 6500}
print(menu.keys())
print(menu.values())
print(menu.items())
print(list(menu.keys()))
```

```
dict_keys(['coffee', 'latte', 'cake'])  ← 키만
dict_values([4500, 5000, 6500])  ← 값만
dict_items([('coffee', 4500), ('latte', 5000), ('cake', 6500)])  ← (키, 값) 튜플
['coffee', 'latte', 'cake']  ← 보통 리스트
```

**keys()**

| "coffee" | "latte" | "cake" |
|---|---|---|
| 4500 | 5000 | 6500 |

**values()**

| "coffee" | "latte" | "cake" |
|---|---|---|
| 4500 | 5000 | 6500 |

**items()**

| "coffee" | "latte" | "cake" |
|---|---|---|
| 4500 | 5000 | 6500 |

3. 둘러보기

# list()로 바꾸면 번호로 꺼낼 수 있어요

[2]
```python
keys = list(menu.keys())
print(keys)
print(keys[0])
print(list(menu.values())[-1])
pair = list(menu.items())[1]
print(pair)
print(pair[0], pair[1])
```
Python

```
['coffee', 'latte', 'cake']
coffee                ← 키 0번
6500                  ← 값 맨 뒤
('latte', 5000)       ← 1번 쌍은 튜플
latte 5000            ← 튜플도 번호로 꺼내요
```

[3]
```python
menu.keys()[0]
```
Python

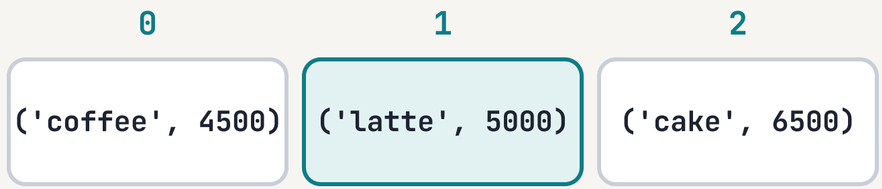
```
TypeError: 'dict_keys' object is not subscriptable
```

번호를 못 붙여요. list()로 먼저 바꿔요

list(menu.keys())




list(menu.items())



3. 둘러보기

# for로 한 쌍씩 꺼내요

[4]
```python
for name in menu:
    print(name)
```

```
coffee
latte
cake
```

← 딕셔너리를 그대로 넣으면 키만 나와요

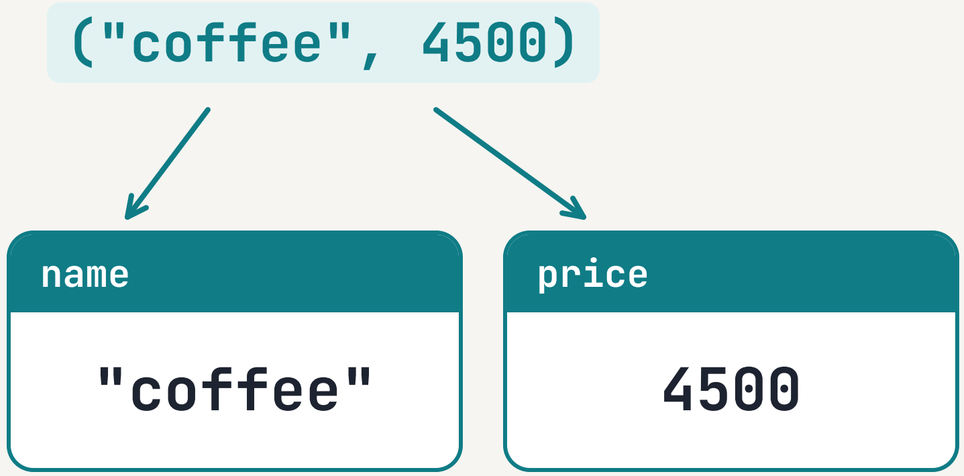
[5]
```python
for name, price in menu.items():
    print(f"{name:<8}{price:>6,}원")
```

```
coffee   4,500원
latte    5,000원
cake     6,500원
```

← name = "coffee", price = 4500

← name = "latte", price = 5000

← name = "cake", price = 6500



3. 둘러보기

# values()에도 sum, max, in을 써요

[6]
```python
print(sum(menu.values()))
print(max(menu.values()))
print(5000 in menu.values())
print(sorted(menu))
```

```
16000   ← 값을 모두 더해요
6500    ← 가장 큰 값
True    ← 값 쪽에서 찾아요
['cake', 'coffee', 'latte']   ← 키만 abc 순으로 정렬해요
```

menu.values()




---

13에서 리스트에 쓰던 함수 그대로예요

`sum([4500, 5000, 6500])`  `sum(menu.values())`  둘 다 16000

3. 둘러보기

# get은 없을 수도 있는 키에 써요

[7]
```python
settings = {"name": "kim"}
print(settings.get("name", "guest"))
print(settings.get("lang", "ko"))
print(settings.get("theme", "light"))
print(settings)
```

```
kim
ko        ← lang이 없어서 기본값
light
{'name': 'kim'}    ← settings는 그대로예요
```

[8]
```python
cart = {"coffee": 2, "tea": 1}
total = 0
for name, qty in cart.items():
    price = menu.get(name, 0)
    print(name, qty, price * qty)
    total += price * qty
print("합계:", total)
```

```
coffee 2 9000
tea 1 0          ← 메뉴판에 없으면 0원
합계: 9000
```

~~price = menu[name]~~ → `price = menu.get(name, 0)` tea에서 KeyError로 멈추지 않아요

3. 둘러보기

# get으로 개수를 세요

[9]
```python
words = "coffee latte coffee cake coffee latte".split()
count = {}
for w in words:
    count[w] = count.get(w, 0) + 1
print(count)
print(count["coffee"])
```

```
{'coffee': 3, 'latte': 2, 'cake': 1}
3
```

[10]
```python
votes = {}
votes["coffee"] = votes["coffee"] + 1
```

```
KeyError: 'coffee'   ← 처음엔 아무 키도 없어요
```

| w | count.get(w, 0) | 넣는 값 |
|---|---|---|
| "coffee" | 0 | 0 + 1 = 1 |
| "latte" | 0 | 0 + 1 = 1 |
| "coffee" | 1 | 1 + 1 = 2 |
| "cake" | 0 | 0 + 1 = 1 |
| "coffee" | 2 | 2 + 1 = 3 |
| "latte" | 1 | 1 + 1 = 2 |

count




참고, 읽기만 · get이 줄인 원래 코드 (if는 17에서)

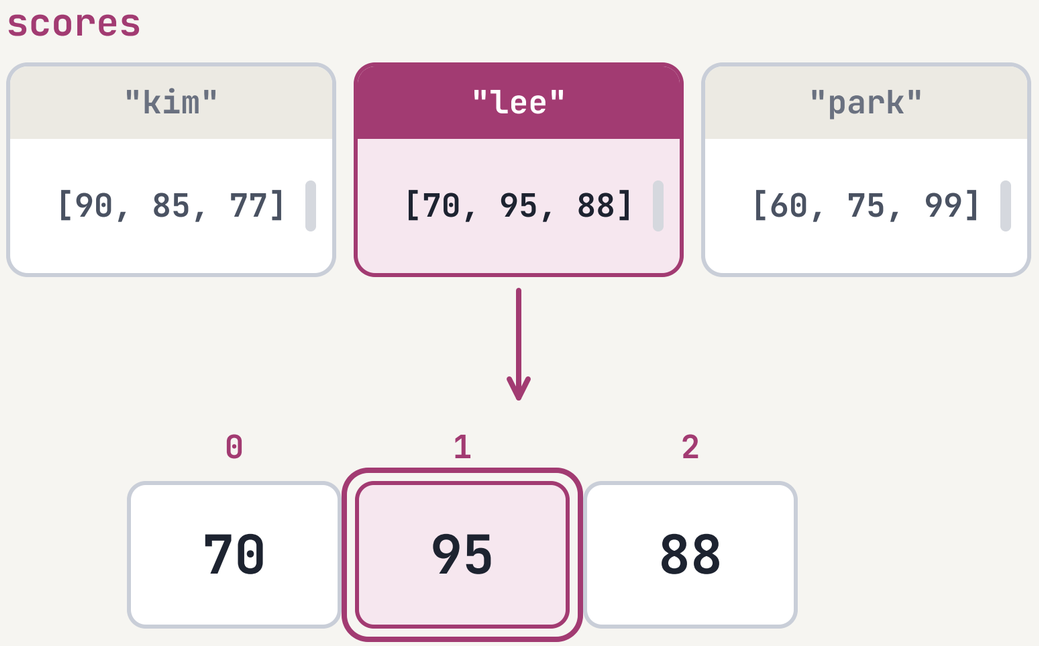
```
count[w] = (count[w] if w in count else 0) + 1
```

4. 리스트와 섞기

# 값에 리스트를 넣을 수 있어요

[1]
```python
scores = {"kim": [90, 85, 77],
          "lee": [70, 95, 88],
          "park": [60, 75, 99]}
print(scores["lee"])
print(scores["lee"][1])
for name, s in scores.items():
    print(name, sum(s), max(s))
```

```
[70, 95, 88]   ← 키로 리스트를 꺼내요
95             ← 그 리스트의 1번
kim 252 90
lee 253 95     ← s가 리스트라 sum, max를 써요
park 234 99
```




14와 같은 점수를 찾을 때

`scores[1][1]`

14 · 1번 학생이 lee라는 걸 외워요

`scores["lee"][1]`

# 딕셔너리를 리스트에 담으면 주문 목록이 돼요

[2]
```python
orders = [{"menu": "coffee", "price": 4500, "qty": 2},
          {"menu": "latte", "price": 5000, "qty": 1},
          {"menu": "cake", "price": 6500, "qty": 3}]
print(len(orders))
print(orders[1])
print(orders[1]["menu"])
print(orders[2]["price"] * orders[2]["qty"])
```
Python

```
3
{'menu': 'latte', 'price': 5000, 'qty': 1}
latte
19500
```
← 바깥은 [번호], 안쪽은 [키]

orders

| | "menu" | "price" | "qty" |
|---|---|---|---|
| 0 | "coffee" | 4500 | 2 |
| 1 | "latte" | 5000 | 1 |
| 2 | "cake" | 6500 | 3 |

13과 비교해요

`orders[1][0]`

13 · 0번이 메뉴라는 걸 외워요

`orders[1]["menu"]`

15 · 키가 뜻을 말해 줘요

# for로 주문을 하나씩 꺼내요

[3]
```python
for order in orders:
    print(order["menu"], order["price"] * order["qty"])
```

```
coffee 9000
latte 5000
cake 19500
```

`order =` `{"menu": "coffee", "price": 4500, "qty": 2}`

`order =` `{"menu": "latte", "price": 5000, "qty": 1}`

`order =` `{"menu": "cake", "price": 6500, "qty": 3}`

orders

| | "menu" | "price" | "qty" |
|---|---|---|---|
| 0 | "coffee" | 4500 | 2 |
| 1 | "latte" | 5000 | 1 |
| 2 | "cake" | 6500 | 3 |

**order에는 주문 딕셔너리 하나가 통째로 들어가요.**

5. 실제로 써 보기

# 리스트, 튜플, 딕셔너리를 나란히 봐요

| | 리스트 | 튜플 | 딕셔너리 |
|---|---|---|---|
| 만들기 | `["latte", 5000]` | `("latte", 5000)` | `{"latte": 5000}` |
| 꺼내기 | `xs[0]` | `t[0]` | `menu["latte"]` |
| 값 바꾸기 | `xs[0] = "tea"` | 못 바꿔요 | `menu["latte"] = 5500` |
| 더하기 | `xs.append("tea")` | 못 더해요 | `menu["tea"] = 4000` |
| 지우기 | `xs.pop(0)` | 못 지워요 | `del menu["tea"]` |
| in이 보는 곳 | 값 | 값 | 키 |

**순서대로 번호로 찾으면 리스트, 이름으로 찾으면 딕셔너리에 담아요.**

5. 실제로 써 보기

# 딕셔너리에서 자주 나는 오류 세 가지

[4]
```python
scores[0]
```
```
KeyError: 0
```

**고치기**

`scores["kim"]`

딕셔너리에는 번호가 없어요

[5]
```python
scores["Lee"]
```
```
KeyError: 'Lee'
```

**고치기**

`scores["lee"]`

키는 대문자와 소문자를 구분해요

[6]
```python
orders["menu"]
```
```
TypeError: list indices must be integers or slices, not str
```

**고치기**

`orders[0]["menu"]`

orders는 리스트예요. 번호부터 적어요

5. 실제로 써 보기

# KANT 카페 영수증을 딕셔너리로 다시 찍어요

[7]

```python
print("=" * 33)
print(f"{'KANT 카페':^31}")
print(f"{'2026-11-03  14:30':^33}")
print("=" * 33)
print(f"{'메뉴명':<7}{'단가':>6}{'수량':>3}{'금액':>8}")
print("-" * 33)
total = 0
count = 0
for order in orders:
    amount = order["price"] * order["qty"]
    print(f"{order['menu']:<10}{order['price']:>8,}{order['qty']:>5}{amount:>10,}")
    total += amount
    count += order["qty"]
print("-" * 33)
print(f"{'수량':<6}{count:>15}")
print(f"{'합계':<6}{total:>25,}")
print("=" * 33)
print(f"{'감사합니다':^28}")
```

Python

```
=================================
          KANT 카페
       2026-11-03  14:30
=================================
메뉴명          단가  수량        금액
---------------------------------
coffee       4,500     2     9,000
latte        5,000     1     5,000
cake         6,500     3    19,500
---------------------------------
수량                       6
합계                          33,500
=================================
          감사합니다
```

13은 for menu, price, qty in orders:, 이번에는 order["price"]처럼 키로 꺼내요

5. 실제로 써 보기

# 장바구니에는 수량만, 가격은 메뉴판에서 찾아요

[8]
```python
menu = {"coffee": 4500, "latte": 5000, "cake": 6500}
cart = {"coffee": 2, "cake": 3}
total = 0
for name, qty in cart.items():
    price = menu[name]
    print(f"{name} {qty}개 {price * qty:,}원")
    total += price * qty
print(f"합계 {total:,}원")
```

```
coffee 2개 9,000원
cake 3개 19,500원
합계 28,500원
```




**가격이 바뀌면 메뉴판 한 곳만 고치면 돼요.**

5. 실제로 써 보기

# 주문 목록에서 메뉴별로 몇 개 팔렸는지 세요

[9]
```python
orders = [{"menu": "coffee", "qty": 2},
          {"menu": "latte", "qty": 1},
          {"menu": "coffee", "qty": 1},
          {"menu": "cake", "qty": 2},
          {"menu": "latte", "qty": 3}]
sold = {}
for order in orders:
    name = order["menu"]
    sold[name] = sold.get(name, 0) + order["qty"]
print(sold)
print(sold["latte"])
print(sum(sold.values()))
```

Python

```
{'coffee': 3, 'latte': 4, 'cake': 2}   ← 메뉴별 개수
4
9   ← 모두 9개
```

**orders**

| | |
|---|---|
| coffee | 2 |
| latte | 1 |
| coffee | 1 |
| cake | 2 |
| latte | 3 |

coffee는 2 + 1

**sold**

| "coffee" | "latte" | "cake" |
|---|---|---|
| 3 | 4 | 2 |

정리

# 막히면 오류의 마지막 줄을 봐요

| 보이는 것 | 까닭 | 고치기 |
|---|---|---|
| `KeyError: 'tea'` | 없는 키를 꺼냈어요 | `menu.get("tea", 0)` |
| `KeyError: 0` | 딕셔너리에서 번호로 꺼냈어요 | `menu["coffee"]` |
| `KeyError: 'Latte'` | 키의 대문자와 소문자가 달라요 | `menu["latte"]` |
| `TypeError: list indices must be integers ...` | 리스트에 키를 넣었어요 | `orders[0]["menu"]` |
| `{'latte': 5500}` | 같은 키를 두 번 넣어 하나만 남았어요 | 키를 다르게 적기 |
| `5500 in menu → False` | in은 키만 봐요 | `5500 in menu.values()` |

**오류가 안 나도 결과가 이상하면 키가 겹치지 않았는지 봐요.**

정리

# 이럴 땐 이렇게 써요

값을 이름으로 담기

```
{"latte": 5000}
```

이름으로 꺼내기

```
menu["latte"]
```

없을 수도 있는 키 꺼내기

```
menu.get("tea", 0)
```

더하거나 바꾸기

```
menu["tea"] = 4000
```

지우기

```
del menu["tea"]   menu.pop("tea")
```

키가 있는지 묻기

```
"tea" in menu
```

한 쌍씩 돌기

```
for k, v in menu.items():
```

개수 세기

```
count[w] = count.get(w, 0) + 1
```

다음 · 16 집합 같은 값은 한 번만 담는 그릇이에요. 딕셔너리의 키처럼 겹치지 않아요.